פרק 15 | רשתות עצביות חוזרות (RNN)

מהרצף והמצב החבוי ועד לאימון **RNN, GRU** ו־**LSTM** — הבנה מתמטית, קוד ופרויקט גמר

מסלול התמחות למידת מכונה ולמידה עמוקה | 5 יחידות לימוד | 30 שעות לפי מפת התכנית במאגר GitHub

גרסת תלמיד מורחבת • המחשות מקוריות • פתרונות הדוגמאות המודרכות בלבד • מעבדה עצמאית ללא פתרונות



אותם סוגי מדידות, אבל הסדר בזמן משנה את משמעותן.

מה נלמד ומדוע?

עד כה למדנו מודלים שמקבלים תמונה, טבלה או וקטור ומתייחסים אליהם כאל דוגמה אחת. בפרק זה נפגוש נתונים שהסדר שלהם חיוני: קריאות חיישן, מילים במשפט, תנועת אצבע על מסך ונתוני מכירות לאורך ימים. נלמד כיצד רשת עצבית יכולה לשמור סיכום פנימי של העבר ולעדכן אותו בכל צעד זמן.

הפרק נשען על הידע מהפרקים הקודמים: וקטורים ומטריצות, פונקציות הפעלה, נגזרות, גרדיאנטים, Backpropagation והערכת מודלים. כאן נוסיף את ממד הזמן ואת הבעיה של תלות בין צעדים רבים.

מטרות ביצועיות

- לזהות סדרה, צעד זמן, חלון קלט, אופק חיזוי ותלות בזמן; להבחין בין Sequence-to-One לבין Sequence-to-Sequence.
- לחשב ידנית שלושה מצבים חבויים, להסביר את השימוש החוזר במשקולות, ולזהות את ממדי המערכים.
- להסביר כיצד BPTT גוזר דרך זמן, לחשב עדכון קטן ולהבחין בין גרדיאנט נעלם לגרדיאנט מתפוצץ.
- לתאר את תפקידי השערים ב־LSTM וב־GRU, ולבחור מודל לפי בעיה וניסוי ולא לפי שם מרשים.
- לבנות ניסוי חיזוי סדרת זמן עם הפרדה כרונולוגית, קו בסיס, אימון, הערכה ותיעוד לקראת פרויקט גמר.

היכן הפרק משתלב בשלוש התכניות?

הקשר לפרויקט	ידע והיקף ההוראה	מסלול
פירוש סדרת מדידות והצעת חקר	קריאת טבלאות וגרפים, חלוקה ל־Train/Test ומודלים בסיסיים	**יחידה 3 — כיתה י׳**
הסבר מתמטי של קלט וגרדיאנט; אין לייחס RNN כחובת יחידה 5	וקטורים, תקנון, הערכה ולידציה כידע קודם	**יחידה 5 — נושאי ליבה**
בניית מסווג או חזאי סדרתי, תיעוד והגנה	RNN — פרק התמחות בהיקף 30 שעות במפת המאגר	**התמחות DL — פרק 15**

שימו לב: ה־GitHub אומתו מול מפת תכנית ההתמחות ב־15 שעות והשיוך לפרק 30. פירוט התוכן והחלוקה הפנימית כאן הם עיבוד דידקטי; אימות מלא מול נוסח ה־PDF הרשמי התקף עדיין נדרש.

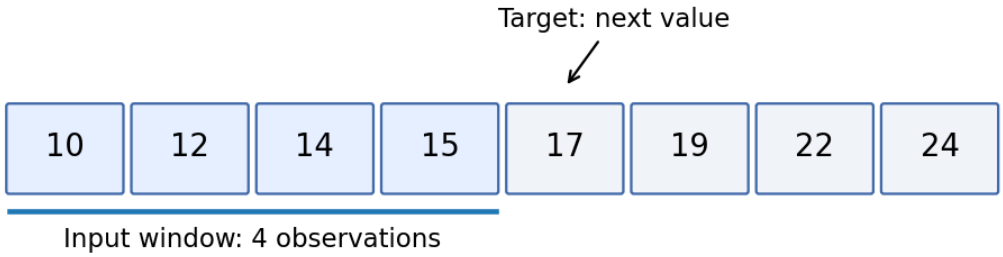
15.1 | מדוע הסדר חשוב? מסיפור על חיישן לשאלת חקר

דודו מנהל מעבדת מחשבים בבית הספר. בכל דקה חיישן מודד את טמפרטורת אחד המחשבים. אם נקבל את המדידות 28, 31, 34, 37 ו־31 מעלות, נחשוד בהתחממות מתמשכת. אך אם אותן מדידות מופיעות בסדר הפוך — 37, 34, 31, 28 — המחשב דווקא מתקרר. הממוצע, המינימום והמקסימום זהים, אולם הסיפור על התהליך שונה לחלוטין.

בנתון טבלאי רגיל, סדר השורות אינו בהכרח משנה את הדוגמה. בסדרה, כל מדידה תלויה במיקום שלה בזמן ובמה שקדם לה. לכן עלינו לשמר את הסדר, ולהימנע משימוש במידע מן העתיד כאשר אנו מבקשים לחזות אותו.

מושגים חדשים

מושג	פירוש	דוגמה
סדרה — Sequence	רצף תצפיות בעלות סדר משמעותי	קריאות חיישן על פני 20 דקות
צעד זמן — Time step	עמדה אחת ברצף, לא בהכרח שנייה אחת	דקה 5 ברצף
חלון — Window	מספר מדידות עוקבות המשמשות קלט	12 הדקות האחרונות
אופק חיזוי — Horizon	כמה צעדים קדימה חוזים	הטמפרטורה בדקה הבאה
Sequence-to-One	רצף קלט לתחזית אחת	האם תופיע תקלה?
Sequence-to-Sequence	פלט לכל צעד או רצף פלט	תווית לכל מילה במשפט



חלון של ארבע מדידות משמש לחיזוי המדידה הבאה. יש לשמור על סדר זמנים.

15.2 | כיצד הופכים סדרת מדידות לדוגמאות אימון?

נניח שקיבלנו שמונה מדידות ברצף. נבחר חלון באורך ארבע. דוגמת האימון הראשונה תכלול את המדידות במקומות 1 – 4 ותחזית היעד שלה תהיה המדידה במקום 5 . הדוגמה השנייה תכלול את המקומות 2 – 5 , והיעד יהיה המקום 6 . כך אנו יוצרים מאותו רצף מספר דוגמאות מבלי לשבור את סדר הזמן.

$$X_1 = [10, 12, 14, 15] \rightarrow y_1 = 17$$

$$X_2 = [12, 14, 15, 17] \rightarrow y_2 = 19$$

חלונות עוקבים חופפים זה לזה. עובדה זו חשובה מאוד בעת החלוקה לקבוצות אימון ובדיקה: אם נערבב חלונות שכנים בין הקבוצות, נתוני הבדיקה עלולים להיות דומים מדי לנתוני האימון ואף לשתף את רוב המדידות. הדרך הבטוחה בדוגמתנו היא לחלק קודם את הרצף לפי זמן, ואז ליצור חלונות בכל קבוצה בנפרד.

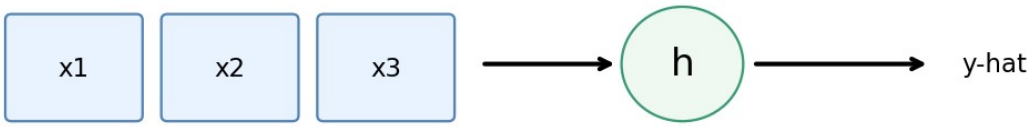
```
def create_windows(values, window=4):
    X, y = [], []
    for end in range(window, len(values)):
        X.append(values[end-window:end])
        y.append(values[end])
    return np.array(X), np.array(y)

values = np.array([10, 12, 14, 15, 17, 19, 22, 24])
X, y = create_windows(values, window=4)
print(X.shape, y.shape)   # (4, 4) (4,)
```

נסו להסביר מדוע מספר החלונות הוא מספר המדידות פחות אורך החלון, כאשר מנבאים את הערך הבא בצעד אחד. מה ישתנה אם נרצה לחזות שני צעדים קדימה?

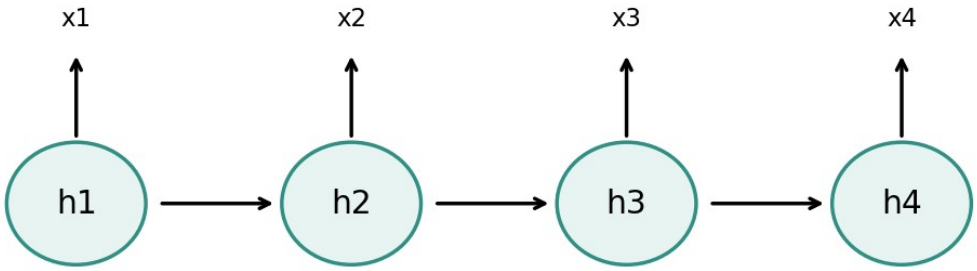
15.3 | מרשת רגילה לרשת עם זיכרון

ברשת Feedforward רגילה החישוב מתקדם מן הקלט לפלט, בלי שמצב פנימי עובר מדוגמה אחת לבאה. ברשת חוזרת קיימת יחידה שמקבלת בכל זמן גם קלט חדש וגם מצב חבוי מן הצעד הקודם. המצב החבוי אינו זיכרון מילולי כמו אצל אדם; זהו וקטור מספרי שנלמד לשמש סיכום שימושי של הרצף שנצפה עד כה.



Sequence model summarizes the ordered inputs

משימת *Sequence-to-One*: כל מדידות החלון מובילות לתחזית אחת.



The same W_x and W_h are reused at each time step

פריסה לאורך זמן: אותו תא *RNN* מופעל כמה פעמים עם אותן משקולות.

הנוסחה הכללית

$$h_t = \tanh(W_x\, x_t + W_h\, h_{t-1} + b_h)$$

$$\hat{y}_t = W_y\, h_t + b_y$$

לנוסחה יש שני חלקים. המכפלה הראשונה מודדת את תרומת הקלט הנוכחי; המכפלה השנייה מודדת את תרומת המצב הקודם; ההטיה מאפשרת הזזה של סכום התרומות. פונקציית tanh מוסיפה אי־ליניאריות ומחזירה ערכים בתחום שבין -1 לבין 1. אותם W_x, W_h ו־b_h משמשים בכל צעדי הזמן — זהו שיתוף משקולות לאורך הזמן.

שימו לב: שימוש באותן משקולות אינו אומר שאותו מצב חבוי מתקבל בכל פעם. הקלט והמצב הקודם משתנים, ולכן גם התוצאה משתנה.

15.4 | דוגמה פתורה: שלושה צעדים וחישוב כל מצב חבוי

כדי שנוכל לחשב ביד, נשתמש במצב חבוי מספרי יחיד. במערכת אמיתית הוא בדרך כלל וקטור בעל כמה רכיבים. נבחר את הפרמטרים הבאים:

$$h_0 = 0, \quad W_x = 0.5, \quad W_h = 0.4, \quad b_h = 0.1$$

$$x_1 = 0.2, \quad x_2 = 0.6, \quad x_3 = 0.4$$

בצעד הראשון אין זיכרון קודם ולכן h_0 שווה אפס. מחשבים את הסכום המשוקלל, מפעילים tanh ושומרים את התוצאה. בצעד השני משתמשים ב־h_1 שחושב בפועל; אין להתחיל בכל צעד מהתחלה.

$$z_1 = 0.5 \cdot 0.2 + 0.4 \cdot 0 + 0.1 = 0.2000$$

$$h_1 = \tanh(0.2000) \approx 0.1974$$

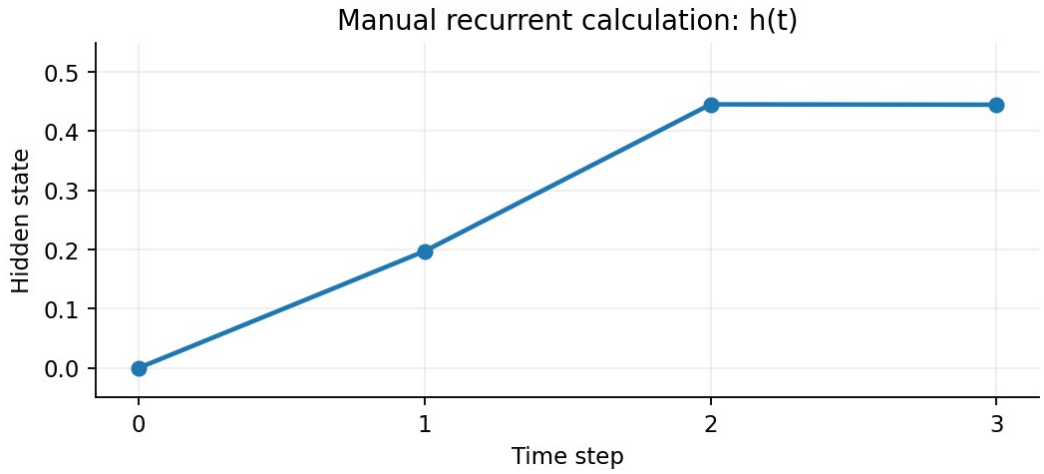
$$z_2 = 0.5 \cdot 0.6 + 0.4 \cdot 0.1974 + 0.1 \approx 0.4790$$

$$h_2 = \tanh(0.4790) \approx 0.4454$$

$$z_3 = 0.5 \cdot 0.4 + 0.4 \cdot 0.4454 + 0.1 \approx 0.4782$$

$$h_3 = \tanh(0.4782) \approx 0.4448$$

צעד	x_t	z_t לפני tanh	h_t אחרי tanh
1	0.2000	0.2000	0.1974
2	0.6000	0.4790	0.4454
3	0.4000	0.4782	0.4448



המצב החבוי משתנה בהדרגה בעקבות הקלט החדש והמצב הקודם.

נוסיף שכבת פלט ליניארית לחיזוי ערך מספרי. נגדיר משקל פלט, הטיה וערך מטרה, ונכתוב אותם בנפרד כדי למנוע בלבול בין הסימונים:

$$W_y = 0.8, \quad b_y = 0.05, \quad y = 0.35$$

$$\hat{y} = 0.8 \cdot h_3 + 0.05 \approx 0.4058$$

$$L = \tfrac{1}{2}(\hat{y} - 0.35)^2 \approx 0.00156$$

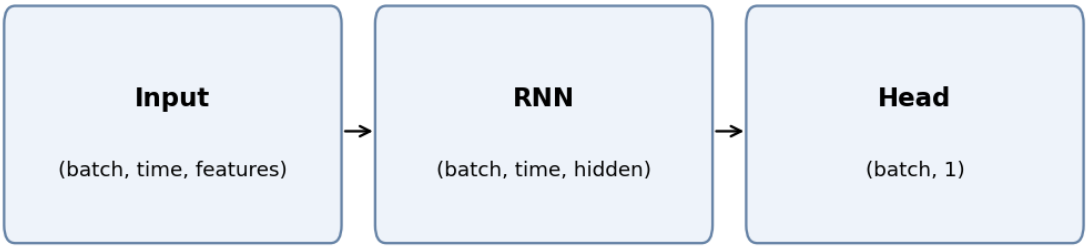
למדנו לבצע Forward Pass שלם. אך בשלב זה המשקולות עדיין קבועות. כדי שהרשת תלמד, נצטרך להבין כיצד שינוי במשקל בכל אחד מצעדי הזמן עשוי לשנות את השגיאה הסופית.

15.5 | ממדים וצורות מערכים — איך המחשב מארגן סדרה?

כאשר מאמנים רשת על כמה סדרות יחד, הקלט אינו וקטור יחיד ואינו מטריצה רגילה של תצפיות. הוא טנזור בשלושה ממדים: מספר דוגמאות באצווה, מספר צעדי זמן, ומספר המאפיינים בכל צעד. כאשר כל רצף מכיל שתים־עשרה דקות ובכל דקה נמדדים שני מאפיינים, דוגמה אחת היא מטריצה ואצווה של דוגמאות היא טנזור. הצורות המדויקות מופיעות בנוסחאות להלן.

$$\texttt{X.shape} = (\text{batch}, \text{sequence_length}, \text{input_size})$$

$$\text{One sample: } (12, 2) \qquad \text{Batch of 32: } (32, 12, 2)$$



כאשר *batch_first=True*, הקלט והשכבות פועלים בסדר (*batch, time, features*).

```
import torch
from torch import nn

X_batch = torch.randn(32, 12, 2)
rnn = nn.RNN(input_size=2, hidden_size=8, batch_first=True)
outputs, last_hidden = rnn(X_batch)
print(outputs.shape)       # (32, 12, 8)
print(last_hidden.shape)   # (1, 32, 8)
```

ממד `8` מציין את מספר הרכיבים במצב החבוי, ולא את מספר המחלקות. הממד הראשון של last_hidden מציין מספר שכבות כפול מספר הכיוונים; כאשר זו שכבה יחידה חד־כיוונית הוא שווה ל־ `1` . תכונת batch_first משנה את צורת הקלט והפלט הסדרתי, אך לא את סדר ממדי המצב החבוי.

15.6 | מהי למידה לאורך זמן? Backpropagation Through Time

ברשת רגילה עקבנו אחרי שגיאה מן הפלט לאחור דרך שכבות. ברשת חוזרת אותו פרמטר מופיע שוב ושוב בצעדי זמן שונים. לכן כאשר מחשבים נגזרת של השגיאה לפי משקל חוזר, יש להביא בחשבון את כל הדרכים שבהן המשקל השפיע על הפלט דרך הצעדים הקודמים. פעולת גזירה זו נקראת Backpropagation Through Time, בקיצור BPTT.

אין צורך לדמיין שיש לנו משקולות חדשות לכל צעד: בפריסה החזותית אנו מציירים עותקים של אותו תא לצורך הבנה, אך המשקולות משותפות. הגרדיאנטים מכל הופעותיהן מצטברים אל אותו פרמטר.

חישוב ידני מפושט של BPTT

כדי לחשב בלי נגזרות ארוכות של tanh, נשתמש בדוגמה לימודית עם הפעלה זהותית. זהו מודל ליניארי פשוט להמחשת כלל השרשרת, לא מודל מומלץ לאימון אמיתי.

$$h_1 = 0.5, \qquad h_2 = w \cdot h_1, \qquad h_3 = 0.5 + w \cdot h_2$$

$$\hat{y} = h_3 = 0.5 + 0.5w^2, \qquad L(w) = \tfrac{1}{2}(\hat{y} - 1)^2$$

נבחר משקל חוזר התחלתי $w=0.2$. נחשב קדימה: $h_2=0.1$, תחזית 0.52 והשגיאה 0.1152. כעת נגזור לאחור. הנגזרת של השגיאה לפי התחזית היא $\hat{y}-1$, והנגזרת של התחזית לפי w היא w. מכפילים אותן לפי כלל השרשרת.

$$dL/dw = (\hat{y} - 1) \cdot w = (0.52 - 1) \cdot 0.2 = -0.096$$

$$w_{new} = w - \eta(dL/dw) = 0.2 - 0.5(-0.096) = 0.248$$

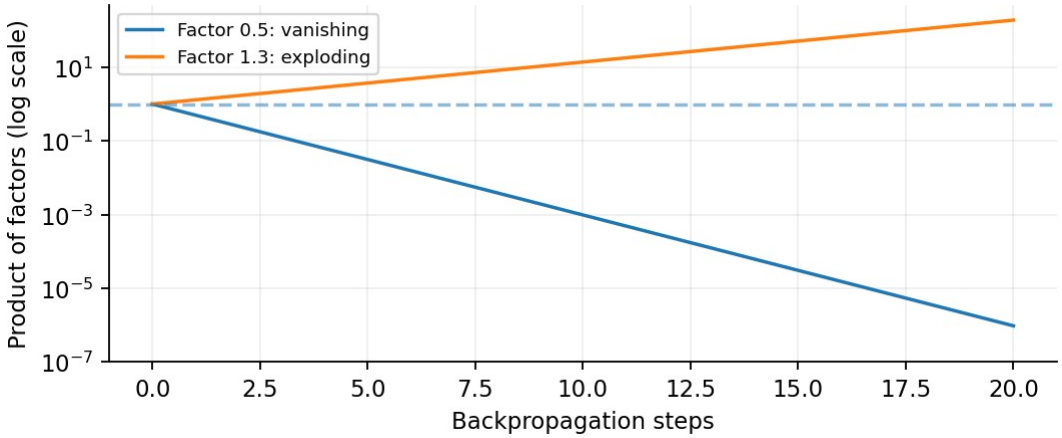
$$\hat{y}_{new} = 0.5 + 0.5 \cdot (0.248)^2 = 0.530752$$

$$L_{new} \approx 0.11010 < 0.11520$$

החישוב מלמד שעדכון פרמטר חוזר משפיע על כמה צעדי זמן יחד. הירידה בשגיאה תקפה בדוגמה ולצעד הזה; אין הבטחה שכל קצב למידה או כל צעד יקטינו תמיד את השגיאה.

15.7 | מדוע קשה לאמן רצפים ארוכים?

ב־BPTT מכפילים נגזרות רבות. כאשר הגורמים קטנים מ־ 1 לאורך צעדים רבים, המכפלה עשויה להתקרב לאפס: זהו גרדיאנט נעלם (Vanishing Gradient). במקרה כזה צעדים מוקדמים מקבלים אות עדכון חלש מאוד, וקשה ללמוד תלות רחוקה. כאשר המכפלות גדולות, הגרדיאנט עלול לגדול מאוד ולגרום לעדכונים לא יציבים: גרדיאנט מתפוצץ (Exploding Gradient).



המחשה מתמטית של מכפלות חוזרות: 0.5 בחזקת t דועך, ואילו 1.3 בחזקת t גדל.

$$0.5^{10} \approx 0.00098, \qquad 1.3^{10} \approx 13.79$$

הדוגמה המספרית מתארת מכפלות פשוטות, ולא טוענת שכל נגזרת בכל RNN היא בדיוק 0.5 או 1.3. ברשת אמיתית התמונה תלויה במשקולות, בפונקציות ההפעלה ובקלט לאורך זמן.

שתי דרכי התמודדות מרכזיות הן שימוש בתאי זיכרון עם שערים (LSTM/GRU), ובמקרה של גרדיאנט מתפוצץ שימוש ב־Gradient Clipping. קיצוץ גרדיאנטים יכול לייצב את הגודל שלהם, אך אינו פותר כשלעצמו בעיית גרדיאנט נעלם.

```
# Training step: clipping happens AFTER backward and BEFORE step.
optimizer.zero_grad()
prediction = model(X_train_batch)
loss = loss_fn(prediction, y_train_batch)
loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
optimizer.step()
```

15.8 | LSTM — תא זיכרון עם שערים

רשת LSTM נועדה לאפשר למידה יעילה יותר של תלות ארוכה. לצד המצב החבוי h_t היא מחזיקה מצב תא c_t המשמש מסלול זיכרון נפרד. שערים לומדים אילו חלקים מן הזיכרון הקודם לשמר, איזה מידע חדש להוסיף ומה לחשוף כפלט. המילה "שער" אינה מתג מכני; זהו וקטור של מספרים בין 0 ל־ 1 , המחושב באמצעות Sigmoid.



השוואה רעיונית: *RNN* פשוט, *GRU* עם שערים, *LSTM* עם מצב תא.

$$f_t = \sigma(W_f[x_t, h_{t-1}] + b_f) \quad \text{(forget gate)}$$

$$i_t = \sigma(W_i[x_t, h_{t-1}] + b_i) \quad \text{(input gate)}$$

$$g_t = \tanh(W_g[x_t, h_{t-1}] + b_g) \quad \text{(candidate)}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t$$

$$o_t = \sigma(W_o[x_t, h_{t-1}] + b_o) \quad \text{(output gate)}$$

$$h_t = o_t \odot \tanh(c_t)$$

הסימון ⊙ פירושו כפל רכיב ברכיב. הסימון [x,h] פירושו שרשור שני וקטורים. הנוסחאות כאן הן ייצוג חינוכי שמאחד קלט ומצב קודם; מימושי ספרייה מסוימים מחזיקים שתי מטריצות נפרדות והטיות נפרדות לכל שער. אין להסיק ש־LSTM תמיד מצליחה או שאין בה לעולם בעיות גרדיאנט.

דוגמה קצרה לשער שכחה

נניח שלמצב התא הקודם ערך 0.8, שער השכחה נתן 0.75, שער הקלט נתן 0.20 והמועמד החדש הוא 0.50. נחשב את הזיכרון המעודכן:

$$c_{new} = 0.75 \cdot 0.8 + 0.20 \cdot 0.5 = 0.60 + 0.10 = 0.70$$

הערך 0.70 משלב מידע שנשמר מן העבר עם מידע חדש. זהו חישוב של צעד אחד בלבד; בפועל הערכים מחושבים על ידי משקולות שנלמדות מהנתונים.

15.9 | GRU — שערים במבנה פשוט יותר

GRU משתמשת בדרך כלל במצב חבוי יחיד ובשני שערים מרכזיים: Reset, המשפיע על האופן שבו העבר נכנס למועמד למצב חדש, ו־Update, הקובע את היחס בין המצב הישן למצב החדש. המבנה פשוט יותר מן המבנה הסטנדרטי של LSTM מבחינת סוגי המצבים, אך אף אחת מהשיטות אינה מנצחת תמיד.

$$h_t = (1 - z_t) \odot n_t + z_t \odot h_{t-1}$$

שימו לב: בנוסחה של PyTorch, ערך z_t גדול נותן משקל גדול יותר לשימור המצב הישן. מקורות אחרים עשויים להגדיר את השער המשלים בכיוון הפוך; לכן יש לקרוא את הנוסחה ולא רק לזכור תיאור מילולי של שער עדכון.

מאפיין	RNN פשוט	GRU	LSTM
מצב עיקרי	h בלבד	h ושערים	h ו־c ושערים

מאפיין	RNN פשוט	GRU	LSTM
התמודדות עם תלות ארוכה	לעיתים קשה	משופרת לעיתים	משופרת לעיתים
מורכבות יחידה	נמוכה יחסית	בינונית	גבוהה יחסית
בחירה מקצועית	Baseline לימודי	ניסוי והשוואה	ניסוי והשוואה

15.10 | משימות סדרתיות נפוצות והחלטת מודל

בבעיית חיזוי סדרת זמן אפשר לקבל חלון של מדידות ולהחזיר ערך אחד — למשל טמפרטורה בדקה הבאה. בעיית Sequence-to-Sequence תחזיר פלט עבור כל זמן, למשל סימון קטעים לא תקינים ברצף. בניתוח טקסט אפשר לסווג משפט שלם או לתייג כל מילה. הבחירה בין משימות תלויה בשאלה שבאמת רוצים לענות עליה, ולא רק בטכנולוגיה הזמינה.

רשת דו־כיוונית יכולה להתחשב הן בעבר והן בעתיד *בתוך רצף שכבר זמין כולו*. אין להשתמש בה בלי זהירות בתחזית בזמן אמת כאשר בעת קבלת ההחלטה המידע מן העתיד עדיין אינו ידוע. זו דוגמה לשגיאה מערכתית שאינה נפתרת על ידי Accuracy גבוה.

15.11 | תהליך ניסוי הוגן לסדרות זמן

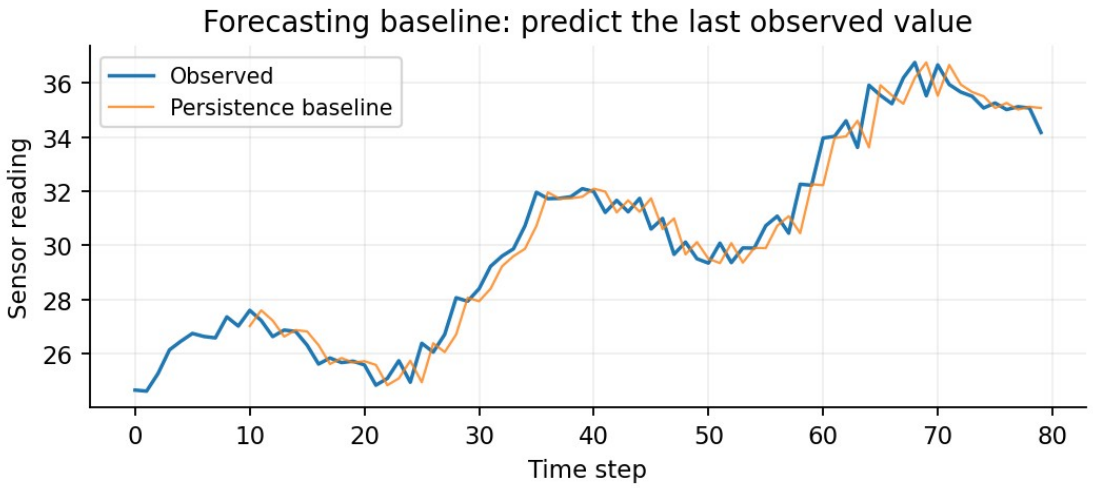
בסדרות זמן סדר החלוקה חשוב במיוחד. לדוגמה, מודל שאמור לפעול בחודש דצמבר צריך להיבחן על תקופה מאוחרת מזו ששימשה לאימונו. אם נערבב נתונים מכל השנה, הסטטיסטיקות של התקופה העתידית עלולות לחדור לאימון. גם תקנון, השלמת חסרים והתאמת פרמטרים צריכים להיעשות רק על נתוני האימון המתאימים.



דוגמה לחלוקה כרונולוגית: אימון בעבר, ולידציה לאחריו, בדיקה סופית בעתיד.

כאשר המערכת אמורה להשתמש במדידות העבר שקדמו לתחילת התקופה החדשה, אפשר לבנות חלונות בהקשר תפעולי מתאים; אך אסור להכניס לחלון קלט ערכים שלא היו ידועים בזמן התחזית. לצורך התרגול נבחר כלל פשוט וזהיר: בונים חלונות בנפרד בכל חלוקה כרונולוגית.

גם אחרי שבנינו RNN, חייבים להשוות אותה לקו בסיס. בחיזוי הערך הבא, אחת האפשרויות הפשוטות היא לנבא שהערך הבא יהיה דומה לערך האחרון שנצפה. מודל מורכב שאינו משפר באופן עקבי את ה־Baseline אינו מוצדק רק משום שהוא רשת עמוקה.



קו בסיס של התמדה (*Persistence*): חיזוי הערך הבא לפי המדידה הקודמת.

15.12 | מודל PyTorch מלא לחיזוי הערך הבא

ניצור נתוני חיישן מדומים בעזרת Python. הנתונים אינם קריאות אמיתיות ממחשבים; כך נוכל ללמוד את התהליך בלי צורך להוריד קובץ חיצוני. תחילה נחלק כרונולוגית, נתקנן באמצעות ממוצע וסטיית תקן מנתוני האימון בלבד, וניצור חלונות באורך 12. 1 2

```
import numpy as np
import torch
from torch import nn

np.random.seed(7)
torch.manual_seed(7)
t = np.arange(600)
sensor = 30 + 0.004*t + 2*np.sin(t/12) + np.random.normal(0, 0.25, len(t))
train_raw = sensor[:390]
val_raw = sensor[390:480]
test_raw = sensor[480:]
mu = train_raw.mean()
sigma = train_raw.std()

def make_windows(raw, window=12):
    scaled = (raw - mu) / sigma
    X, y = [], []
    for end in range(window, len(scaled)):
        X.append(scaled[end-window:end])
        y.append(scaled[end])
    X = torch.tensor(np.array(X), dtype=torch.float32).unsqueeze(-1)
    y = torch.tensor(np.array(y), dtype=torch.float32).unsqueeze(-1)
    return X, y

X_tr, y_tr = make_windows(train_raw)
X_va, y_va = make_windows(val_raw)
X_te, y_te = make_windows(test_raw)
print(X_tr.shape, y_tr.shape)
```

הפונקציה unsqueeze מוסיפה ממד אחד של מאפיינים. כך צורת הקלט הופכת מ־(n_windows,12) לצורה (n_windows,12,1), כפי ש־RNN עם batch_first=True מצפה לקבל.

הגדרת המודל ואימון בסיסי

```
class SensorRNN(nn.Module):
    def __init__(self, hidden_size=12):
        super().__init__()
        self.rnn = nn.RNN(input_size=1, hidden_size=hidden_size,
                          batch_first=True)
        self.head = nn.Linear(hidden_size, 1)
```

```
    def forward(self, x):
        sequence_output, hidden = self.rnn(x)
        last_step = sequence_output[:, -1, :]
        return self.head(last_step)

model = SensorRNN(hidden_size=12)
loss_fn = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

for epoch in range(25):
    model.train()
    optimizer.zero_grad()
    pred = model(X_tr)
    loss = loss_fn(pred, y_tr)
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    optimizer.step()
    model.eval()
    with torch.no_grad():
        val_loss = loss_fn(model(X_va), y_va).item()
    if (epoch + 1) % 5 == 0:
        print(epoch + 1, round(loss.item(), 4), round(val_loss, 4))
```

אנחנו מאמנים דוגמה לימודית על אצווה אחת בכל Epoch כדי שקוד האימון יהיה קריא. בפרויקט גדול נשתמש לרוב ב־DataLoader ובאצוות קטנות. תוצאת המודל תלויה גם במימוש, בזרע האקראי ובנתונים; אסור לטעון מראש שהוא טוב יותר מקו בסיס.

הערכה ביחידות המקוריות

```
model.eval()
with torch.no_grad():
    pred_scaled = model(X_te).squeeze().numpy()
actual_scaled = y_te.squeeze().numpy()
pred_real = pred_scaled * sigma + mu
actual_real = actual_scaled * sigma + mu

rmse_model = np.sqrt(np.mean((pred_real - actual_real)**2))
baseline_real = X_te[:, -1, 0].numpy() * sigma + mu
rmse_baseline = np.sqrt(np.mean((baseline_real-actual_real)**2))
print('RNN RMSE:', round(rmse_model, 3))
print('Persistence RMSE:', round(rmse_baseline, 3))
```

מדוע מחזירים את התחזיות ליחידות המקוריות? משום שקל יותר להסביר לתלמיד או למשתמש שגיאת חיזוי של 0.6 מעלות מאשר 0.4 יחידות תקן. ההשוואה הוגנת רק אם שני המודלים נבדקו על אותם יעדים בזמן. אל תחזרו להתאים את המודל על Test לאחר צפייה בתוצאות.

15.13 | השוואת RNN, GRU ו־LSTM באופן מבוקר

כדי לבדוק האם תאי זיכרון משפרים את התוצאות, נשאיר את אותו מאגר, חלוקה, חלון, מדד ומספר Epochs. נשנה רק את סוג התא. כך נוכל להסביר מה הייתה השאלה הניסויית ואיזה גורם שינינו. מספר הפרמטרים עשוי להיות שונה ולכן אין לומר שהשוואה כזו ״משווה אותו מספר פרמטרים״; אם דרושה בדיקה מדויקת יותר יש להתאים את המודלים בהתאם.

```
def make_recurrent(kind, in_features=1, hidden=12):
    layers = {'RNN': nn.RNN, 'GRU': nn.GRU, 'LSTM': nn.LSTM}
    recurrent = layers[kind](input_size=in_features, hidden_size=hidden,
                             batch_first=True)
    return recurrent

for kind in ['RNN', 'GRU', 'LSTM']:
    net = make_recurrent(kind)
    print(kind, sum(p.numel() for p in net.parameters()))
```

הקטע מציג השוואה של מספר פרמטרים בתא בלבד. בפרויקט המלא צריך להוסיף גם שכבת פלט, לבצע אימון של כל מודל באותם תנאים ולשמור תוצאות מסודרות. המודל שהשיג את ציון הולידציה הטוב ביותר לא בהכרח ינצח בכל מאגר אחר.

15.14 | טעויות נפוצות, אתיקה והגנה בעל־פה

- לערבב את כל חלונות הזמן באקראי בלי לבחון תלות כרונולוגית; במקרה זה ייתכן שהערכת הביצועים אופטימית מדי.
- לחשב StandardScaler על כלל המדידות לפני חלוקת Train/Validation/Test; זו זליגת מידע.
- לבלבל בין ממד האצווה לממד הזמן, או לקחת את hidden האחרון בצורה שאינה תואמת למספר השכבות/הכיוונים.
- להניח ש־LSTM תמיד טובה יותר מ־RNN/GRU, בלי בדיקה מול Baseline וביצועי הולידציה.
- להשתמש בתווית או במדד מן העתיד כמפתח לחיזוי העבר; לעיתים המקור ל״מודל מצוין״ הוא מידע שלא היה זמין בפועל.
- לשמור רק Accuracy או RMSE בלי לתעד תאריכים, פרמטרים, גרסאות והחלטות מקצועיות.

בפרויקטים על נתוני אנשים יש להימנע מהסקת מסקנות לא מוצדקות על אדם מתוך דפוס התנהגות קצר. בסדרות ניטור רפואיות, למשל, הפלט צריך לעבור בחינה מקצועית והערכת סיכונים. גם במעבדת בית ספר יש להבהיר שהמודל מסייע לקבלת החלטה ואינו מחליף אבחון תקלה אמיתי.

15.15 | תרגול מדורג — ידע, חישוב וחשיבה

רמת יסוד

- 1 . הסבירו כיצד שינוי סדר המדידות יכול לשנות את המשמעות בלי לשנות את הממוצע.
- 2 . עבור רצף בן 5 1 מדידות וחלון באורך 4 , כמה דוגמאות לחיזוי הצעד הבא ניתן ליצור? נמקו.
- 3 . ציינו מה מייצג כל ממד בטנזור שצורתו (3 , 0 1 , 0 2), כאשר batch_first=True.
- 4 . הגדירו מהו מצב חבוי והסבירו מדוע הוא אינו שווה בהכרח לתצפית האחרונה.

רמת תרגול מתמטית

- 5 . חשבו שלושה צעדי RNN עם h0=0, Wx=0.2, Wh=0.5, b=0, x=[1,0,1] והפעלה זהותית; השוו ל־tanh.
- 6 . עבור $L(w)=\frac{1}{2}(0.5+0.5w^2-1)^2$ ו־w=0.4, מצאו נגזרת ובצעו עדכון בקצב 1 . 0 .
- 7 . חשבו 0 . 8 בחזקת 1 2 ופירשו מה היא ממחישה על מכפלות גרדיאנטים.
- 8 . ב־LSTM: c_prev=0.4, f=0.6, i=0.2, g=0.5. חשבו c_new ופרשו את שני האיברים.

רמת בגרות, חקר ופרויקט

- 9 . קיבלתם נתוני חיישן לפי תאריך. תכננו חלוקת Train/Validation/Test והסבירו היכן תקנון צריך להתבצע.

- 10. מודל RNN השיג RMSE=0.8 ו־Baseline השיג RMSE=0.6 על Test. האם ההחלטה לבחור ב־RNN מוצדקת? הציעו שני ניסויי המשך.
- 11. דוגמה מציגה ביצועי אימון מעולים אך ולידציה חלשה. הסבירו אפשרויות: התאמת יתר, שינוי התפלגות ומידע חסר.
- 12. הציעו תכנית למערכת לניטור מחשבים: מאפיינים, חלון, יעד, מדד הצלחה, תוצר תוכנה וקריטריון בטיחות.

15.16 | מעבדת Colab — משימות עצמאיות

לצד פרק זה קיימת מחברת Jupyter עם 12 משימות הדרגתיות. חלק מהתאים כוללים סימוני TODO או סימני שאלה כקוד להשלמה — אלה מקומות שבהם התלמיד אמור לכתוב בעצמו. פתרונות מלאים נשמרים לגרסת המורה בלבד.

תוצר ההגשה כולל מחברת פועלת, השוואת Baseline ו־RNN על Test כרונולוגי, גרף תחזיות, טבלת ניסויים ופסקת מסקנות הכוללת לפחות מגבלה אחת. אין חובה לקבל ציון מסוים; חובה להצדיק את התהליך.

15.17 | תכנית הוראה מומלצת ל־30 שעות

מפגש	מוקד	תוצר ביצועי
1 — 3 שעות	רצף, חלון, יעד והבדל מטבלה	טבלת חלונות ידנית
2 — 3 שעות	ארכיטקטורה, שיתוף משקולות ומצב חבוי	תרשים RNN מחושב
3 — 3 שעות	Forward Pass ו־NumPy	שלושה צעדים מחושבים
4 — 3 שעות	BPTT וכלל השרשרת	נגזרת ועדכון ידניים
5 — 3 שעות	Vanishing/Exploding ו־Clipping	גרפים ופירוש
6 — 3 שעות	LSTM ושערים	חישוב מצב תא
7 — 3 שעות	GRU והשוואת ארכיטקטורות	טבלת בחירה מנומקת
8 — 3 שעות	חלונות, חלוקה בזמן ו־Baseline	Pipeline נתונים
9 — 3 שעות	אימון RNN ב־PyTorch והערכה	מודל וגרף תחזיות
10 — 3 שעות	ניסוי מבוקר, כתיבה והגנה	מחברת ודוח קצר

זו חלוקה פדגוגית מוצעת לעשרה מפגשים בני שלוש שעות. את שעות ההתמחות והיקף הדרישה בפועל יש לקבוע על פי תכנית בית הספר והמסמך הרשמי העדכני.

15.18 | הכנה לעבודת הגמר — תיק חקר מינימלי

שאלה שהתלמיד חייב לענות עליה	רכיב
מהם הנתונים, למי הם שייכים ומתי נאספו?	מקור נתונים

שאלה שהתלמיד חייב לענות עליה	רכיב
איזו תחזית מבצעים ומה היה ידוע בזמן התחזית?	**הגדרת יעד**
מדוע נבחר אורך הרצף הזה?	**בחירת חלון**
איזה פתרון פשוט חייבים לנצח?	**Baseline**
מה נבדק: RNN, GRU או LSTM, ומה נשאר קבוע?	**מודלים**
כיצד נבחרו פרמטרים בלי לראות Test?	**ולידציה**
מה גודל הטעות במונחים של המשתמש?	**מדדים**
כיצד מזינים רצף חדש למערכת?	**תוכנה**
מה עלול להשתבש בפעילות בעולם האמיתי?	**מגבלות**

15.19 | מילון מושגים

הגדרה בעברית	מונח
רשת המעדכנת מצב חבוי לאורך רצף	**Recurrent Neural Network**
וקטור פנימי המסכם מידע מהעבר	**Hidden State**
פריסה חזותית של אותו תא לאורך זמן	**Unrolling**
גזירת השגיאה לאחור לאורך צעדי זמן	**BPTT**
היחלשות אות העדכון כאשר נגזרות מוכפלות	**Vanishing Gradient**
גדילת גרדיאנט עד לחוסר יציבות	**Exploding Gradient**
תא חוזר עם מצב זיכרון ושערים	**LSTM**
תא חוזר עם שערי איפוס ועדכון	**GRU**
חלון הקלט ואופק התחזית	**Window / Horizon**
זליגת מידע שאינו זמין בזמן התחזית	**Temporal Leakage**
ניבוי הערך הבא לפי הערך האחרון שנצפה	**Persistence Baseline**

15.20 | מבחן מסכם — האם אתם מבינים רשת סדרתית?

להלן מבחן עצמי קצר לפני המעבדה האחרונה. התשובות אינן מופיעות בגרסת התלמיד; מומלץ להסביר כל בחירה במילים ולא רק לכתוב מספר. אפשר להגיש תשובות בכתב או להציגן בהסבר בן חמש דקות מול תלמיד אחר.

כיצד תראו הבנה	שאלה מסכמת	מוקד
הסבר לפי מצב חבוי	שני רצפים מכילים אותם ערכים בסדר שונה. האם ה־RNN חייב	**סדר ונתונים**

כיצד תראו הבנה	שאלה מסכמת	מוקד
	להחזיר אותו פלט?	
נימוק לכל אחד מהממדים	.X.shape=(32,12,2), hidden_size=8 מה צורת הפלט הסדרתי?	**ממדיות**
חיבור ל־BPTT	מה המשמעות של שיתוף המשקולות לאורך 1 2 צעדים?	**אימון**
ניסוי הוגן ובסיס השוואה	מדוע לא נכון להבטיח ש־LSTM תמיד תנצח GRU?	**מודלים**
מסקנה מקצועית ומגבלה	הדיוק על Test היה חלש יותר מ־Baseline. מה תדווחו?	**בדיקה**

משימת מעבר לפרויקט: הכינו כרטיס ניסוי בן חצי עמוד הכולל הגדרת בעיה, סוג רצף, אורך חלון, חלוקה כרונולוגית, Baseline, מדד מרכזי ומשפט אחד המסביר מתי השימוש במודל לא יהיה אחראי.

15.21 | מקורות קריאה והבחנה בין חובה להרחבה

עוגן תכנית ההתמחות: מאגר Data-Science-ML-Course, פרק 15 — רשתות סדרתיות חוזרות (RNN), 30 שעות. הקישור למיקום בפרק מאמת את שם הפרק והיקפו במפה; הוא אינו מחליף קריאה של מסמך משרד החינוך הרשמי המלא.

ספרות מקצועית: Zhang, Lipton, Li & Smola — Dive into Deep Learning, פרקי RNN ו־BPTT; Simon Prince — Understanding Deep Learning; Goodfellow, Bengio & Courville — Deep Learning; Aurélien Géron — Hands-On Machine Learning. שיטות ההסבר הותאמו ונכתבו מחדש באופן מקורי לתלמידי תיכון.

תיעוד יישום: PyTorch nn.RNN, nn.GRU, nn.LSTM ו־torch.nn.utils.clip_grad_norm_. יש לקרוא את הגרסה המותקנת כאשר עובדים עם נתוני תלמידים וגרסאות תוכנה שונות.

קישור לחומרי המקור במאגר: https://github.com/samiabo75/Data-Science-ML-Course/tree/main/03-Deep-Learning-5-Units/15-Recurrent-Neural-Networks

D2L: https://en.d2l.ai/chapter_recurrent-neural-networks/index.html

PyTorch RNN: https://docs.pytorch.org/docs/stable/generated/torch.nn.RNN.html

PyTorch GRU: https://docs.pytorch.org/docs/stable/generated/torch.nn.GRU.html

PyTorch LSTM: https://docs.pytorch.org/docs/stable/generated/torch.nn.LSTM.html